AUTONOMOUS AI TECHNICAL INTERVIEWER (RAG v2.1)

Master Architecture Document: Pipeline Mechanics, Terminology, Mathematical Geometry & Evaluation Rubric

Architecture: Hybrid Dual-Tier RAG | **Vector DB:** FAISS IndexFlatIP | **Embedding:** all-MiniLM-L6-v2 (d=384, L_2 Norm) | **Judge:** Gemini Flash Lite (Single-Pass)

1. Master RAG Pipeline Execution Flow & Architecture

The table below outlines the 7 sequential stages of the Retrieval-Augmented Generation (RAG) pipeline, specifying the component, underlying algorithm, output, and operational latency at each layer:

Phase	Component	Technology / Underlying Algorithm	Output / Artifact	Latency
1. Ingestion	Hybrid Tier Store	Tier-1 Local JSON DB + Tier-2 GitHub API Crawler & GFG DOM Parser	Clean Markdown / Code blocks	0ms (Hit) / ~1.2s
2. Chunking	Sliding Window	1000-character chunk size with 200-character boundary overlap	Overlapped text segments + metadata	< 5ms
3. Embedding	SentenceTransformer	all-MiniLM-L6-v2 (d=384) + Euclidean L_2 Normalization	Unit vectors $\mathbf{u} \in \blacksquare^{384}$ ($\|\|u\|\|_2 = 1.0$)	< 15ms
4. Indexing	Vector Store	faiss.IndexFlatIP (Exact Inner Product on normalized vectors)	In-memory similarity matrix space	< 1ms
5. Retrieval	Cosine Similarity	Top-K (K=5) search with ≥ 0.52 threshold routing & auto-caching	Top-5 verified technical context blocks	< 2ms
6. Synthesis	Prompt Augmentation	Context bounding, anti-hallucination fences & dynamic caching	Context-grounded challenge question	~1.0s
7. Evaluation	Single-Pass LLM Judge	4-part 10-pt weighted rubric + co-generated adaptive follow-up probe	10-pt assessment report + instant probe	~1.8s – 2.7s (0.0s follow-up)

2. Deep Technical Breakdown: Every Phase, Term & Algorithm Explained

Phase 1: Ingestion & Hybrid Dual-Tier Discovery

• **Data Ingestion:** The extraction, sanitization, and structured ingestion of raw technical text from external repositories into the pipeline.
• **Hybrid Dual-Tier Store:** An offline-first architecture. It checks a local FAISS vector index (Tier-1) for zero network latency (0ms). If the topic is missing, it dynamically triggers live online discovery (Tier-2) across GitHub and GeeksforGeeks, then caches the new topic into offline_db/dynamic_cache.json for instant future hits.
• **GitHub REST API (v3) & Query Expansion:** Expands queries into interview permutations (e.g. *'topic coding interview'*) and ranks candidate repositories using a weighted scoring formula: $\text{Score} = 0.6(\text{Stars}) + 0.2(\text{Forks}) + 0.2(\text{Relevance})$.
• **In-Memory ZIP Streaming:** Streams repository ZIP archives directly into RAM via io.BytesIO and decompresses them with zipfile.ZipFile, eliminating disk write bottlenecks.
• **DOM Parsing & BeautifulSoup Crawler:** Crawls GeeksforGeeks articles, parses the Document Object Model (DOM) HTML tree, and strips advertisements, headers, and sidebars, preserving clean technical theory and code.

Phase 2: Chunking & Text Segmentation

• **Chunking:** Dividing long continuous text into discrete segments to adhere to transformer token limits and preserve semantic density.
• **Sliding Window Algorithm:** A segmentation algorithm where a window of length $W = 1000\text{ characters}$ moves forward with a stride $S = 800\text{ characters}$.
• **Boundary Overlap (200 characters):** Consecutive chunks share 200 characters of text. This guarantees that multi-line algorithmic invariants (e.g., boundary condition if left <= right: or recursive base cases) are never split across chunk boundaries.
• **Chunk Metadata Enrichment:** Attaches file origin (rel_path), line offsets, and topic attribution to every chunk for full verification traceability.

Phase 3: Vector Embeddings & Mathematical Geometry

• **Vector Embedding:** A continuous multi-dimensional projection where semantic language concepts translate into geometric coordinates.
• **SentenceTransformer (all-MiniLM-L6-v2):** A 6-layer MiniLM neural network mapping text to a 384-dimensional dense space ($\mathbb{R}^{384}$).
• **Euclidean L_2 Normalization:** Raw vectors $\vec{e}$ are scaled to unit length so that $|\vec{u}|_2 = 1.0$:

L_2 Normalization Formula: $\vec{u} = \frac{\vec{e}}{|\vec{e}|_2} = \frac{\vec{e}}{\sqrt{\sum_{i=1}^{384} e_i^2}} \implies ||\vec{u}||_2 = 1.0$

Phase 4: Indexing & Vector Search Architecture (FAISS)

• **FAISS (Facebook AI Similarity Search):** A high-performance C++ vector library utilizing hardware-accelerated BLAS matrix operations.
• **faiss.IndexFlatIP:** An exact, uncompressed index computing Inner Product ($\vec{q} \cdot \vec{d}$) without quantization loss.
• **Cosine Similarity ≡ Inner Product Equivalence:** Standard Cosine Similarity is defined as $\frac{\vec{q} \cdot \vec{d}}{|\vec{q}|_2 |\vec{d}|_2}$. Because all vectors are L_2-normalized to unit length ($|\vec{q}|_2 = 1.0, |\vec{d}|_2 = 1.0$), the denominator equals 1.0, proving that:

```
Inner Product Equivalence:      $\text{Cosine Similarity}(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{384} (q_i \times d_i)$
```

Phase 5: Retrieval, Cosine Similarity & Threshold Routing

• **Semantic Retrieval:** Matches underlying conceptual meaning rather than literal keyword strings (e.g. *'FIFO queue'* matches *'Breadth First Search traversal'*).
• **Top-K Retrieval ($K = 5$):** Extracts the 5 nearest neighbor chunks with highest cosine similarity.
• **Similarity Threshold Routing ($\theta = 0.52$):** If $\max(\text{Score}) \ge 0.52$, the system triggers a Tier-1 local hit. If $\max(\text{Score}) < 0.52$, it triggers Tier-2 dynamic online search.
• **Dynamic Self-Learning (Auto-Caching):** Newly crawled online material is chunked, embedded, and appended to offline_db/dynamic_cache.json, turning future queries into instant offline hits.

Phase 6: Synthesis, Context Bounding & Prompt Augmentation

• **Prompt Augmentation:** Injects the Top-5 retrieved source chunks into the LLM system prompt context window.
• **Context Bounding & Anti-Hallucination Guardrails:** Sets strict negative prompt constraints (*'Formulate challenge question based ONLY on retrieved reference context; do not extrapolate or invent facts'*).
• **Cross-Source Synthesis:** Reconciles practical implementation details (GitHub) with conceptual theory (GeeksforGeeks) into a balanced challenge.

Phase 7: LLM-as-a-Judge Evaluation & Single-Pass Probing

• **LLM-as-a-Judge:** An automated evaluator using Gemini Flash Lite to objectively grade candidate answers against ground truth.
• **Single-Pass Co-generation:** The evaluator prompt produces the full 10-point evaluation report **and** the adaptive follow-up probe in the **same single API call**, reducing follow-up wait latency to **0.0s**.
• **Speculative Multi-Turn Prefetching:** Follow-up answers are evaluated while speculatively pre-generating the subsequent probe inside a FollowUpFeedback container, eliminating wait time on multi-turn iterations.
• **Model Tiering & Instant 503 Failover:** Prioritizes gemini-flash-lite-latest with immediate failover on 503 errors, reducing round-trip latency from $\sim 95\text{s} \rightarrow 1.0\text{s} - 1.8\text{s}$.

3. Candidate Evaluation Parameters & Weighted Rubric

Candidate responses are judged strictly against the retrieved technical ground truth across **4 explicit weighted parameters** totaling **10.0 points**:

Parameter Dimension	Weight	Max Pts	Evaluation Criteria & Focus
1. Conceptual Correctness & Invariants	40%	4.0 / 10	Selection of appropriate data structures (e.g., Queue vs Stack for BFS, Hash Map + DLL for LRU Cache); preserving core algorithmic invariants without anti-patterns.
2. Completeness, Edge Cases & Constraints	25%	2.5 / 10	Boundary conditions (left ≤ right), cycle handling via visited set, null/empty collections, duplicate values, memory leak prevention, recursion base cases.
3. Complexity Analysis (Time & Space)	20%	2.0 / 10	Accurate Big-O asymptotic runtime (O(V+E), O(log n)); correct distinction between auxiliary space (call stack frames) vs total space.
4. Technical Precision & Communication	15%	1.5 / 10	Correct engineering terminology, clear non-contradictory logic, sound justification for design tradeoffs.

Score-to-Verdict Mapping:
• **Score ≥ 7.5 (Verdict: Correct):** Meets core invariants $\rightarrow$ triggers **advanced deep-dives** (0-1 BFS, scale optimizations).
• **Score 5.0 – 7.4 (Verdict: Partially Correct):** Approach sound, but missed an edge case $\rightarrow$ triggers a **targeted invariant probe**.
• **Score < 5.0 (Verdict: Incorrect):** Fundamental misconception detected $\rightarrow$ triggers **foundational remedial correction**.

4. Latency Optimizations & Performance Benchmarks

By replacing multi-second retry backoffs and eliminating sequential round-trip calls, total turnaround latency was reduced by **~35x to 50x**:

Pipeline Operation	Previous Latency	Optimized Latency	Performance Gain
LLM API Round-Trip	22.0s – 95.3s (gemini-3.6-flash queue)	**1.0s – 1.6s** (flash-lite)	**~50x Speedup**

Initial Answer Evaluation	22.0s – 95.0s	**~2.7s**	**~35x Speedup**
First Follow-up Probe Display	3.0s – 5.0s (sequential call)	**0.0s (Instant Co-generated)**	**Latency Eliminated**
Follow-up Answer Review	15.0s – 30.0s	**~1.8s**	**~15x Speedup**
Multi-turn Next Probe Display	3.0s – 5.0s (sequential call)	**0.0s (Instant Prefetched)**	**Latency Eliminated**